HUNDRED DAYS OF CODE
DAY 32

QUESTION: Write a C program to perform the following menu-driven operations using separate functions:

1. Get two variable length 2D arrays and prove that matrix addition is commutative.
2. Get two variable length 2D arrays and prove that matrix multiplication is not commutative.
3. Exit

SOLUTION:

1. Aim

To write a menu-driven C program using **separate functions and variable-length 2D arrays** to:

1. Verify that matrix addition is **commutative**, i.e. A + B = B + A.
2. Demonstrate that matrix multiplication is **not commutative in general**, i.e. A × B ≠ B × A.
3. Exit the program.

2. Algorithm

Main Algorithm

1. Start the program.
2. Display the menu:
    - Prove matrix addition is commutative.
    - Demonstrate matrix multiplication is not commutative.
    - Exit.
3. Accept the user's choice.
4. Use a switch statement to call the appropriate function.
5. Repeat the menu until the user selects Exit.
6. Stop.

Operation 1: Matrix Addition

HUNDRED DAYS OF CODE
DAY 32

1. Input the number of rows and columns.
2. Declare two variable-length matrices A and B of the same dimensions.
3. Input the elements of both matrices.
4. Calculate A + B.
5. Calculate B + A.
6. Compare the corresponding elements of both resulting matrices.
7. If all elements are equal, display that matrix addition is commutative.
8. Display both resulting matrices.

Operation 2: Matrix Multiplication

1. Input the dimensions of matrices A and B.
2. For both A × B and B × A to exist, use square matrices of the same order in this program.
3. Input the elements of matrices A and B.
4. Calculate A × B using three nested loops.
5. Calculate B × A using three nested loops.
6. Compare the corresponding elements of both resulting matrices.
7. If they are different, demonstrate that matrix multiplication is not commutative for the entered matrices.
8. Display both products.

C PROGRAM:

```
#include <stdio.h>


// Function declarations
void matrixAddition();
void matrixMultiplication();


int main()
```

HUNDRED DAYS OF CODE
DAY 32

```
{
   int choice;

   do
   {
      printf("\n----- MATRIX OPERATIONS MENU -----\n");
      printf("1. Prove Matrix Addition is Commutative\n");
      printf("2. Demonstrate Matrix Multiplication is Not Commutative\n");
      printf("3. Exit\n");

      printf("Enter your choice: ");
      scanf("%d", &choice);

      switch(choice)
      {
         case 1:
            matrixAddition();
            break;

         case 2:
            matrixMultiplication();
            break;

         case 3:
            printf("Program exited successfully.\n");
            break;
```

HUNDRED DAYS OF CODE
DAY 32

```
        default:
            printf("Invalid choice! Please try again.\n");
      }

   } while(choice != 3);

   return 0;
}



// Function for Matrix Addition
void matrixAddition()
{
   int rows, cols;
   int i, j;
   int equal = 1;

   printf("\nEnter number of rows and columns: ");
   scanf("%d %d", &rows, &cols);

   // Variable Length Arrays
   int A[rows][cols];
   int B[rows][cols];
   int AB[rows][cols];
   int BA[rows][cols];
```

HUNDRED DAYS OF CODE
DAY 32

```
printf("\nEnter elements of Matrix A:\n");


for(i = 0; i < rows; i++)
{
   for(j = 0; j < cols; j++)
   {
      scanf("%d", &A[i][j]);
   }
}

printf("\nEnter elements of Matrix B:\n");


for(i = 0; i < rows; i++)
{
   for(j = 0; j < cols; j++)
   {
      scanf("%d", &B[i][j]);
   }
}


// Calculate A + B and B + A
for(i = 0; i < rows; i++)
{
   for(j = 0; j < cols; j++)
   {
```

HUNDRED DAYS OF CODE
DAY 32

```
      AB[i][j] = A[i][j] + B[i][j];
      BA[i][j] = B[i][j] + A[i][j];
   }
}

printf("\nA + B:\n");

for(i = 0; i < rows; i++)
{
   for(j = 0; j < cols; j++)
   {
      printf("%d\t", AB[i][j]);
   }
   printf("\n");
}

printf("\nB + A:\n");

for(i = 0; i < rows; i++)
{
   for(j = 0; j < cols; j++)
   {
      printf("%d\t", BA[i][j]);
   }
   printf("\n");
}
```

HUNDRED DAYS OF CODE
DAY 32

```
    // Compare A+B and B+A
    for(i = 0; i < rows; i++)
    {
        for(j = 0; j < cols; j++)
        {
            if(AB[i][j] != BA[i][j])
            {
                equal = 0;
            }
        }
    }

    if(equal == 1)
    {
        printf("\nA + B = B + A\n");
        printf("Hence, matrix addition is commutative.\n");
    }
}



// Function for Matrix Multiplication
void matrixMultiplication()
{
    int n;
    int i, j, k;
```

HUNDRED DAYS OF CODE
DAY 32

```
int equal = 1;


printf("\nEnter the order of the square matrices: ");
scanf("%d", &n);


// Variable Length Arrays
int A[n][n];
int B[n][n];
int AB[n][n];
int BA[n][n];


printf("\nEnter elements of Matrix A:\n");


for(i = 0; i < n; i++)
{
   for(j = 0; j < n; j++)
   {
      scanf("%d", &A[i][j]);
   }
}


printf("\nEnter elements of Matrix B:\n");


for(i = 0; i < n; i++)
{
   for(j = 0; j < n; j++)
```

HUNDRED DAYS OF CODE
DAY 32

```
  {
    scanf("%d", &B[i][j]);
  }
}

// Calculate A x B
for(i = 0; i < n; i++)
{
  for(j = 0; j < n; j++)
  {
    AB[i][j] = 0;

    for(k = 0; k < n; k++)
    {
      AB[i][j] += A[i][k] * B[k][j];
    }
  }
}

// Calculate B x A
for(i = 0; i < n; i++)
{
  for(j = 0; j < n; j++)
  {
    BA[i][j] = 0;
```

HUNDRED DAYS OF CODE
DAY 32

```
      for(k = 0; k < n; k++)
      {
         BA[i][j] += B[i][k] * A[k][j];
      }
   }
}

printf("\nA x B:\n");

for(i = 0; i < n; i++)
{
   for(j = 0; j < n; j++)
   {
      printf("%d\t", AB[i][j]);
   }
   printf("\n");
}

printf("\nB x A:\n");

for(i = 0; i < n; i++)
{
   for(j = 0; j < n; j++)
   {
      printf("%d\t", BA[i][j]);
   }
```

HUNDRED DAYS OF CODE
DAY 32

```
    printf("\n");
}

// Compare AB and BA
for(i = 0; i < n; i++)
{
    for(j = 0; j < n; j++)
    {
        if(AB[i][j] != BA[i][j])
        {
            equal = 0;
        }
    }
}

if(equal == 0)
{
    printf("\nA x B != B x A\n");
    printf("Hence, matrix multiplication is not commutative "
           "for the given matrices.\n");
}
else
{
    printf("\nA x B = B x A for the given matrices.\n");
    printf("These matrices are a special commuting case.\n");
}
```

HUNDRED DAYS OF CODE
DAY 32

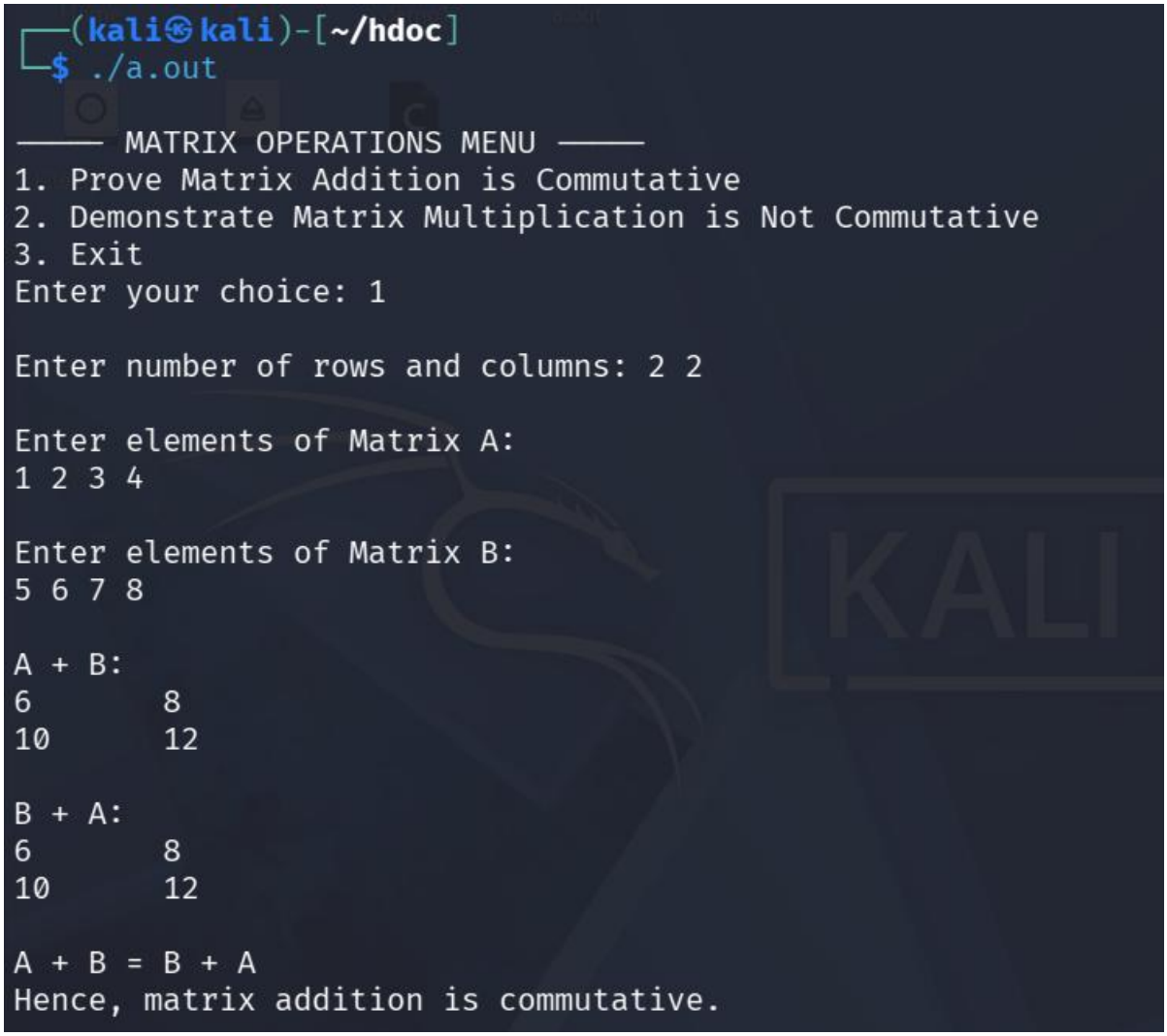
}

TEST CASE TABLE:

Test Case	Operation	Input	Expected Result
1	Addition	A = [1 2; 3 4], B = [5 6; 7 8]	A+B = B+A
2	Addition	A = [2 0; 1 3], B = [4 5; 2 1]	Addition is commutative
3	Multiplication	A = [1 2; 3 4], B = [5 6; 7 8]	AB ≠ BA
4	Multiplication	A = identity matrix, B = any 2×2 matrix	AB = BA special case
5	Exit	Choice = 3	Program terminates
6	Invalid	Choice = 5	Displays invalid choice

COMPILATION AND EXECUTION:

HUNDRED DAYS OF CODE
DAY 32



HUNDRED DAYS OF CODE
DAY 32

